

Relatório Técnico: Sistemas Operacionais

Integrantes:

- Luiz Filipe Costa Assis - 202265188AC
- Thamiris Balbi - 201865116AC
- Wesley Santos de Lima - 202465558C

6.1 - Contador compartilhado

Código disfuncional

Na versão inicial, ocorre uma condição de corrida: as threads tentam alterar o contador ao mesmo tempo. Como a operação não é instantânea, uma thread acaba por ler e gravar valores desatualizados, corrompendo o resultado final.

Código funcional

Para corrigir, a variável foi declarada como `std::atomic<int>`. Isto torna a alteração atômica (indivisível), aplicando o princípio da exclusão mútua. Assim, o sistema obriga uma thread a esperar pela outra, evitando sobreposições e garantindo o resultado correto (zero).

```
● bash-5.1$ g++ S0a1.cpp -o S0a1_errado -pthread
● bash-5.1$ ./S0a1_errado
● Valor final: 0
● bash-5.1$
```

6.2 - Produtor-consumidor com buffer limitado

Código disfuncional

Foi realizada uma versão da resolução do problema sem a utilização de semáforos propositalmente, para ilustração e visualização do erro. Sem eles, os sinais de que o buffer está vazio ou cheio podem ser perdidos no meio do caminho, podendo gerar condições de deadlock. Foi observado principalmente a condição de corrida, levando a threads que atropelam outras nas ações e resultados inesperados. Foram testados para $N = 5$, $N = 10$, $N = 1000$ e $N = 10000$, para uma thread de cada e também para dois produtores e dois consumidores.

```
[CONSUMIDOR] Pegou: 54
[CONSUMIDOR] Pegou: 43
[CONSUMIDOR] Pegou: 63
[CONSUMIDOR] Pegou:
[PRODUTOR] Colocou: 23
[PRODUTOR] Colocou: 28
[PRODUTOR] Colocou: 15
[PRODUTOR] Colocou: 2
[PRODUTOR] Colocou: 48
[PRODUTOR] Colocou: 2
[PRODUTOR] Colocou: 79
[PRODUTOR] Colocou: 34
[PRODUTOR] Colocou: 88
[PRODUTOR] Colocou: 26
[PRODUTOR] Colocou: 19
[PRODUTOR] Colocou: 35
[PRODUTOR] Colocou: 53
[PRODUTOR] Colocou: 46
[PRODUTOR] Colocou: 79
[PRODUTOR] Colocou: 18
42
[CONSUMIDOR] Pegou: 79
[CONSUMIDOR] Pegou: 18
[CONSUMIDOR] Pegou: 36
[CONSUMIDOR] Pegou: 41
[CONSUMIDOR] Pegou: 46
[CONSUMIDOR] Pegou: 79
[CONSUMIDOR] Pegou: 18
[CONSUMIDOR] Pegou: 36
[CONSUMIDOR] Pegou: 41
[CONSUMIDOR] Pegou: 46
```

Teste com várias threads, $N = 5$

```
consumidor acordou
0 item: 75 foi produzido
contador em produtor: 2
0 item: 32 foi produzido
contador em produtor: 3
0 item: 100 foi produzido
contador em produtor: 4
0 item: 4 foi produzido
contador em produtor: 5
Produtor dormiu
Produtor dormiu
Produtor dormiu
Produtor dormiu
Produtor dormiu
Produtor dormiu
contador em consumidor: 4
produtor acordou
Produtor dormiu
0 item: 52 foi produzido
contador em produtor: 5
Consumidor processou o item: 75
contador em consumidor: 4
produtor acordou
Consumidor processou o item: 32
contador em consumidor: 3
Consumidor processou o item: 100
Produtor dormiu
0 item: 77 foi produzido
contador em produtor: 3
0 item: 43 foi produzido
```

Teste com uma thread, $N = 5$

```

[CONSUMIDOR] Pegou: 80
[CONSUMIDOR] Pegou: 98
[CONSUMIDOR] Pegou: 88
[CONSUMIDOR] Pegou: 39
[CONSUMIDOR] Pegou: 70
[CONSUMIDOR] Pegou: 84
[CONSUMIDOR] Pegou: 41
[PRODUTOR] Colocou: 24
[PRODUTOR] Colocou: 98
[PRODUTOR] Colocou: 97
[PRODUTOR] Colocou: 95
[PRODUTOR] Colocou: 2
[PRODUTOR] Colocou: 54
[PRODUTOR] Colocou: 21
[PRODUTOR] Colocou: 4
[CONSUMIDOR] Pegou: 96
[CONSUMIDOR] Pegou: 67
[PRODUTOR] Colocou: 73
[PRODUTOR] Colocou: 21
[CONSUMIDOR] Pegou: 34
[CONSUMIDOR] Pegou: 45
[CONSUMIDOR] Pegou: 60
[CONSUMIDOR] Pegou: 22[CONSUMIDOR] Pegou: 63
[CONSUMIDOR] Pegou: 59
[CONSUMIDOR] Pegou: 44
[CONSUMIDOR] Pegou: 69
[CONSUMIDOR] Pegou: 78

```

```

[CONSUMIDOR] Pegou: 9
[CONSUMIDOR] Pegou: 45[PRODUTOR] Colocou: 68
[PRODUTOR] Colocou: 54
[PRODUTOR] Colocou: 11
[PRODUTOR] Colocou: 79
[PRODUTOR] Colocou: 2
[PRODUTOR] Colocou: 13
[PRODUTOR] Colocou: 37
[PRODUTOR] Colocou: 74
[PRODUTOR] Colocou: 20

[CONSUMIDOR] Pegou: 58[PRODUTOR] Colocou: 100

[CONSUMIDOR] Pegou: 6
6
[CONSUMIDOR] Pegou: 32
[CONSUMIDOR] Pegou: 79
[CONSUMIDOR] Pegou: 24
[CONSUMIDOR] Pegou: 62
[CONSUMIDOR] Pegou: 61
[CONSUMIDOR] Pegou: [CONSUMIDOR] Pegou: 98
[CONSUMIDOR] Pegou: 87
[CONSUMIDOR] Pegou: 75
[CONSUMIDOR] Pegou: 93
[CONSUMIDOR] Pegou: 27

```

Teste com várias threads, $N = 10000$, e teste com uma thread com $N = 10000$

Código funcional

Foram implementados semáforos pelos conceitos de mutexes para exclusão mútua juntamente com variáveis de condição para executar a sincronização, para o consumidor pegar itens quando tem disponíveis, bem como o produtor preencher o buffer quando há espaço, sem a perda de sinal. Também é resolvida a questão da

condição de corrida que havia anteriormente, fazendo as soluções ficarem imprevisíveis.

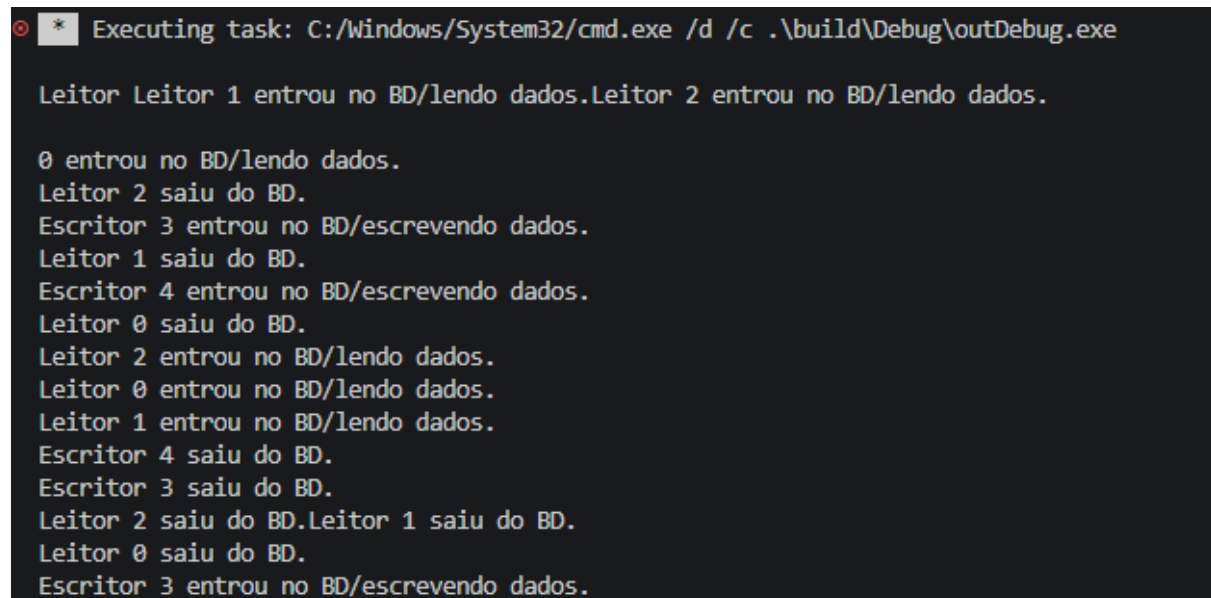
[CONSUMIDOR] Pegou: 71	[CONSUMIDOR] Pegou: 91
[PRODUTOR] Colocou: 81	[CONSUMIDOR] Pegou: 36
[PRODUTOR] Colocou: 78	[PRODUTOR] Colocou: 40
[PRODUTOR] Colocou: 16	[PRODUTOR] Colocou: 49
[PRODUTOR] Colocou: 56	[PRODUTOR] Colocou: 66
[PRODUTOR] Colocou: 17	[PRODUTOR] Colocou: 63
[PRODUTOR] Colocou: 54	[PRODUTOR] Colocou: 69
[PRODUTOR] Colocou: 35	[PRODUTOR] Colocou: 20
[PRODUTOR] Colocou: 53	[PRODUTOR] Colocou: 70
[PRODUTOR] Colocou: 8	[PRODUTOR] Colocou: 83
[PRODUTOR] Colocou: 80	[PRODUTOR] Colocou: 14
[CONSUMIDOR] Pegou: 81	[CONSUMIDOR] Pegou: 40
[CONSUMIDOR] Pegou: 78	[CONSUMIDOR] Pegou: 49
[CONSUMIDOR] Pegou: 16	[CONSUMIDOR] Pegou: 66
[CONSUMIDOR] Pegou: 56	[CONSUMIDOR] Pegou: 63
[CONSUMIDOR] Pegou: 17	[CONSUMIDOR] Pegou: 69
[CONSUMIDOR] Pegou: 54	[CONSUMIDOR] Pegou: 20
[CONSUMIDOR] Pegou: 35	[CONSUMIDOR] Pegou: 70
[CONSUMIDOR] Pegou: 53	[CONSUMIDOR] Pegou: 83
[CONSUMIDOR] Pegou: 8	[CONSUMIDOR] Pegou: 14
[CONSUMIDOR] Pegou: 80	[PRODUTOR] Colocou: 82
[PRODUTOR] Colocou: 63	[PRODUTOR] Colocou: 17
[PRODUTOR] Colocou: 27	[PRODUTOR] Colocou: 78
[PRODUTOR] Colocou: 4	[PRODUTOR] Colocou: 74
[PRODUTOR] Colocou: 60	[PRODUTOR] Colocou: 18
[PRODUTOR] Colocou: 66	[PRODUTOR] Colocou: 79
[PRODUTOR] Colocou: 88	[PRODUTOR] Colocou: 4
[PRODUTOR] Colocou: 10	[PRODUTOR] Colocou: 82
	[PRODUTOR] Colocou: 24

Teste com uma thread de cada, N = 10000 Teste com duas threads de cada, N = 10000

6.3 Leitores e Escritores:

Código disfuncional:

No exemplo do código disfuncional podemos observar que escritores e leitores conseguem atuar juntos no banco de dados, podendo gerar inconsistência e corrupção nos mesmos como observamos pelo print dos logs a seguir:



```
Executing task: C:/Windows/System32/cmd.exe /d /c .\build\Debug\outDebug.exe

Leitor Leitor 1 entrou no BD/lendo dados.Leitor 2 entrou no BD/lendo dados.

0 entrou no BD/lendo dados.
Leitor 2 saiu do BD.
Escritor 3 entrou no BD/escrevendo dados.
Leitor 1 saiu do BD.
Escritor 4 entrou no BD/escrevendo dados.
Leitor 0 saiu do BD.
Leitor 2 entrou no BD/lendo dados.
Leitor 0 entrou no BD/lendo dados.
Leitor 1 entrou no BD/lendo dados.
Escritor 4 saiu do BD.
Escritor 3 saiu do BD.
Leitor 2 saiu do BD.Leitor 1 saiu do BD.
Leitor 0 saiu do BD.
Escritor 3 entrou no BD/escrevendo dados.
```

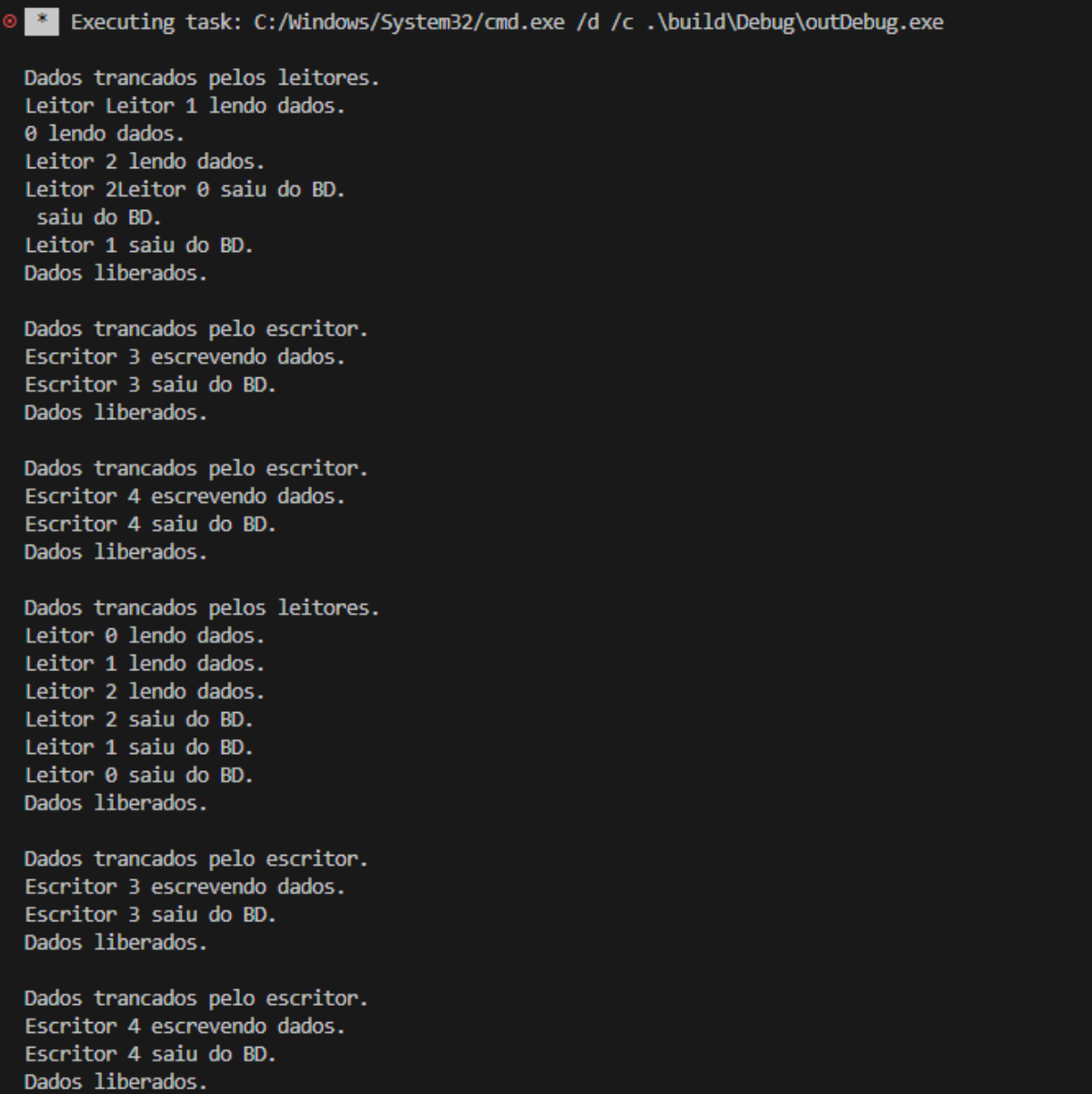
Figura 1: Logs código incorreto leitores e escritores.

Isso se dá pela falta de um sistema que garanta a exclusão mútua de cada integrante, causando o acesso à área crítica(BD) simultânea. Além disso, não há mutex que garanta a consistência do contador de leitores, deixando em condição de corrida.

Código funcional:

A solução proposta adotou o uso de 2 mutexes simples, um controlando o acesso ao contador de leitores e outro controlando o acesso ao banco de dados. Sobre o mutex do BD, cada leitor confere se é o primeiro, e se for, fecha o acesso ao BD para escritores, do contrário ele apenas adentra a área crítica e lê os dados. Os escritores não precisavam de um contador por sua natureza exclusiva, e apenas fechavam o acesso ao BD sempre que necessitavam escrever algo.

Essa implementação possui uma preocupação que é a de que um escritor pode ter que esperar muito até que tenha acesso ao BD, e segue em anexo seu log:



```
Executing task: C:/Windows/System32/cmd.exe /d /c .\build\Debug\outDebug.exe

Dados trancados pelos leitores.
Leitor Leitor 1 lendo dados.
0 lendo dados.
Leitor 2 lendo dados.
Leitor 2Leitor 0 saiu do BD.
saiu do BD.
Leitor 1 saiu do BD.
Dados liberados.

Dados trancados pelo escritor.
Escritor 3 escrevendo dados.
Escritor 3 saiu do BD.
Dados liberados.

Dados trancados pelo escritor.
Escritor 4 escrevendo dados.
Escritor 4 saiu do BD.
Dados liberados.

Dados trancados pelos leitores.
Leitor 0 lendo dados.
Leitor 1 lendo dados.
Leitor 2 lendo dados.
Leitor 2 saiu do BD.
Leitor 1 saiu do BD.
Leitor 0 saiu do BD.
Dados liberados.

Dados trancados pelo escritor.
Escritor 3 escrevendo dados.
Escritor 3 saiu do BD.
Dados liberados.

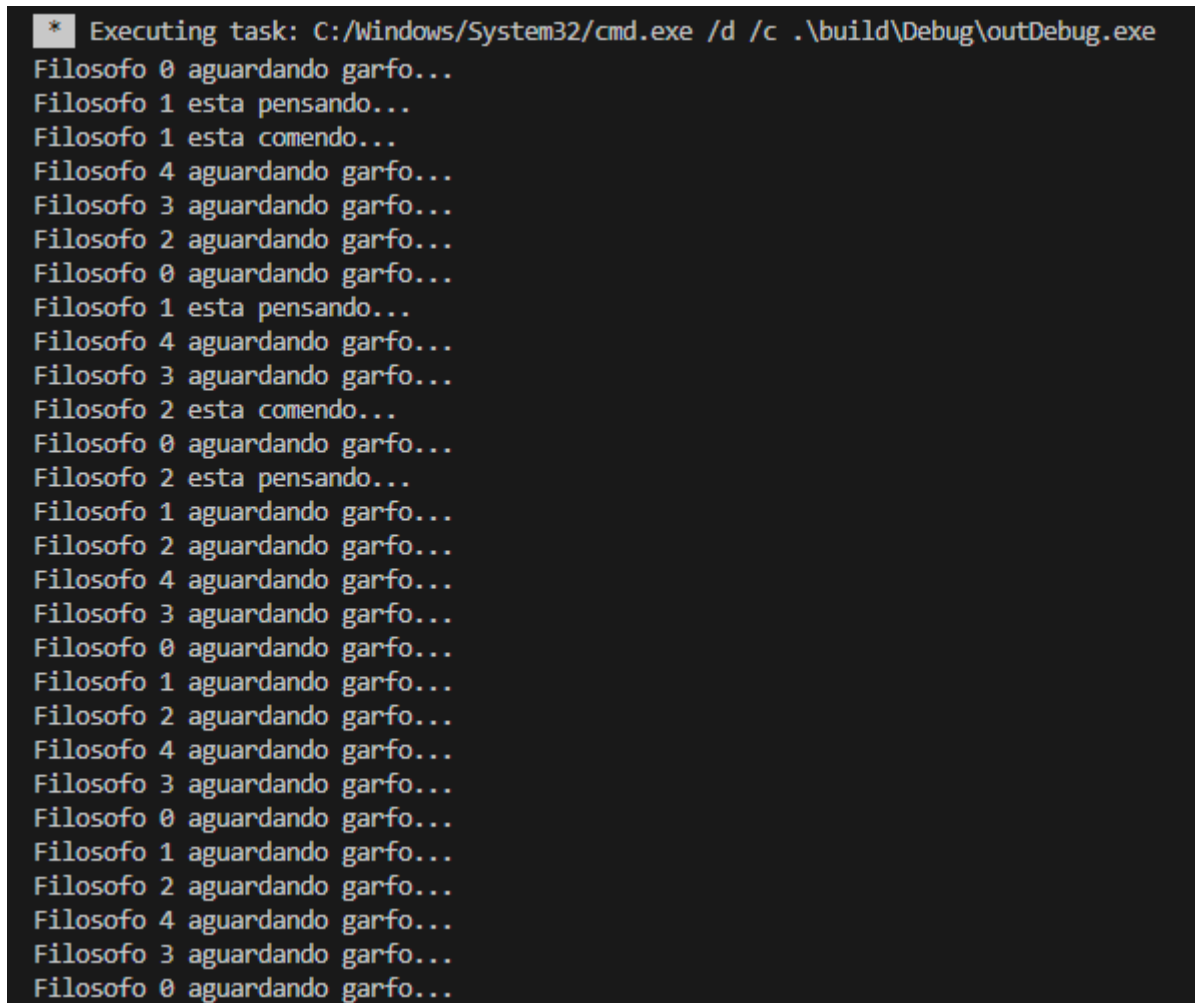
Dados trancados pelo escritor.
Escritor 4 escrevendo dados.
Escritor 4 saiu do BD.
Dados liberados.
```

Figura 2: Logs código corrigido leitores e escritores.

6.4 Jantar dos Filósofos:

Código disfuncional:

No exemplo do código disfuncional é evidente as possibilidades de se cair em um deadlock, ou livelock, a depender do sincronismo de cada thread na hora de pegar os garfos, com deadlocks ocorrendo basicamente em todas as simulações feitas para esse problema, como podemos ver nos prints:



```
* Executing task: C:/windows/System32/cmd.exe /d /c .\build\Debug\outDebug.exe
Filosofo 0 aguardando garfo...
Filosofo 1 esta pensando...
Filosofo 1 esta comendo...
Filosofo 4 aguardando garfo...
Filosofo 3 aguardando garfo...
Filosofo 2 aguardando garfo...
Filosofo 0 aguardando garfo...
Filosofo 1 esta pensando...
Filosofo 4 aguardando garfo...
Filosofo 3 aguardando garfo...
Filosofo 2 esta comendo...
Filosofo 0 aguardando garfo...
Filosofo 2 esta pensando...
Filosofo 1 aguardando garfo...
Filosofo 2 aguardando garfo...
Filosofo 4 aguardando garfo...
Filosofo 3 aguardando garfo...
Filosofo 0 aguardando garfo...
Filosofo 1 aguardando garfo...
Filosofo 2 aguardando garfo...
Filosofo 4 aguardando garfo...
Filosofo 3 aguardando garfo...
Filosofo 0 aguardando garfo...
Filosofo 1 aguardando garfo...
Filosofo 2 aguardando garfo...
Filosofo 4 aguardando garfo...
Filosofo 3 aguardando garfo...
Filosofo 0 aguardando garfo...
```

Figura 1: Logs código incorreto jantar dos filósofos.

```
* Executing task: C:/Windows/System32/cmd.exe /d /c .\build\Debug\outDebug.exe
Filosofo 1 esta pensando...
Filosofo 1 esta comendo...
Filosofo 2 aguardando garfo...
Filosofo 3 aguardando garfo...
Filosofo 0 aguardando garfo...
Filosofo 1 esta pensando...
Filosofo 4 aguardando garfo...
Filosofo 1 esta comendo...
Filosofo 2 aguardando garfo...
Filosofo 3 aguardando garfo...
Filosofo 0 aguardando garfo...
Filosofo 1 esta pensando...
Filosofo 4 aguardando garfo...
Filosofo 1 aguardando garfo...
Filosofo 2 aguardando garfo...
Filosofo 3 aguardando garfo...
Filosofo 0 aguardando garfo...
Filosofo 4 aguardando garfo...
Filosofo 1 aguardando garfo...
Filosofo 3 aguardando garfo...
Filosofo 2 aguardando garfo...
Filosofo 0 aguardando garfo...
Filosofo 4 aguardando garfo...
Filosofo 1 aguardando garfo...
Filosofo 2 aguardando garfo...
Filosofo 3 aguardando garfo...
Filosofo 0 aguardando garfo...
Filosofo 4 aguardando garfo...
```

Figura 2: Logs código incorreto jantar dos filósofos.


```
* Executing task: C:/Windows/System32/cmd.exe /d /c .\build\Debug\outDebug.exe
Filosofo 2 aguardando garfo...
Filosofo 1 aguardando garfo...
Filosofo 0 aguardando garfo...
Filosofo 4 aguardando garfo...
Filosofo 3 esta pensando...
Filosofo 3 esta comendo...
Filosofo 2 aguardando garfo...
Filosofo 1 aguardando garfo...
Filosofo 4 aguardando garfo...
Filosofo 0 aguardando garfo...
Filosofo 3 esta pensando...
Filosofo 2 aguardando garfo...
Filosofo 1 aguardando garfo...
Filosofo 3 aguardando garfo...
Filosofo 0 aguardando garfo...Filosofo 4 aguardando garfo...

Filosofo 2 aguardando garfo...
Filosofo 1Filosofo 3 aguardando garfo...
    aguardando garfo...
Filosofo 4 aguardando garfo...
Filosofo 0 aguardando garfo...
Filosofo 2 aguardando garfo...
Filosofo 0 aguardando garfo...
Filosofo 4 aguardando garfo...
Filosofo 3 aguardando garfo...
Filosofo 1 aguardando garfo...
Filosofo 2 aguardando garfo...
Filosofo 4 aguardando garfo...
```

Figura 3: Logs código incorreto jantar dos filósofos.

Essas são 3 execuções seguidas, em cada uma levou entre 20 e 40 segundos para cair em situação de deadlock, isso se dá pela falha lógica no acesso aos garfos, tendo todas as condições necessárias para um deadlock. Por mais que haja um mutex controlando o acesso aos garfos para manter a integridade dos dados, a falha de lógica continua presente.

Código funcional:

A solução proposta ataca a propriedade de posse e espera que gera o deadlock, a ideia é proteger os garfos com o mutex e obrigar os filósofos a tentarem pegar os 2 garfos ao mesmo tempo, e caso não consigam, dormem por um período aleatório de tempo. A solução, por mais que simples e resolvedora dos deadlocks, apresenta dois problemas principais: o risco de starvation (caso o filósofo se encontre em uma sincronização ruim) e um gargalo no desempenho na mesa, pois como o mutex controla toda a operação, apenas um filósofo tenta pegar os garfos por vez. Seguem os logs da execução do código:

```
* Executing task: C:/Windows/System32/cmd.exe /d /c .\build\Debug\outD
Filosofo Filosofo 1 esta pensando...Filosofo 3 esta pensando...

Filosofo 4 esta pensando...
0 esta pensando...
Filosofo 2 esta pensando...
Filosofo Filosofo 1 esta comendo...
4 esta comendo...
Filosofo 2 aguardando garfos...
Filosofo 0 aguardando garfos...
Filosofo 3 aguardando garfos...
Filosofo 4 esta pensando...
Filosofo 1 esta pensando...
Filosofo 4 esta comendo...
Filosofo 1 esta comendo...
Filosofo 3 aguardando garfos...
Filosofo 2 aguardando garfos...
Filosofo 0 aguardando garfos...
Filosofo 1 esta pensando...
Filosofo 4 esta pensando...
Filosofo 1 esta comendo...
Filosofo 4 esta comendo...
Filosofo 3 aguardando garfos...
Filosofo 0 aguardando garfos...
Filosofo 2 aguardando garfos...
Filosofo 4 esta pensando...
Filosofo 1 esta pensando...
Filosofo 1 esta comendo...
Filosofo 4 esta comendo...
Filosofo 2 aguardando garfos...Filosofo 3 aguardando garfos...

Filosofo 0 aguardando garfos...
Filosofo 4 esta pensando...
Filosofo 1 esta pensando...
Filosofo 3 esta comendo...Filosofo 0 esta comendo...

Filosofo 2 aguardando garfos...
Filosofo 1 aguardando garfos...
Filosofo 4 aguardando garfos...
Filosofo 3 esta pensando...
Filosofo 0 esta pensando...
Filosofo 3 esta comendo...
Filosofo 0 esta comendo...
Filosofo 2 aguardando garfos...
Filosofo 4 aguardando garfos...
```

Figura 4: Logs código corrigido jantar dos filósofos.

6.5 - Barbeiro Sonolento

Código disfuncional:

Foi implementado com o uso de variáveis comuns propositalmente e por vezes, observado os erros de sinal perdido, condição de corrida devido ao conflito de sincronização entre os sinais, desperdício de cpu com o busy waiting. Ocorreria também possível deadlock caso ocorresse uma situação de impasse.

```
[BARBEIRO] Cortando cabelo... ha 0 clientes esperando.  
Sala vazia: barbeiro vai dormir  
Cliente 11 chegou e sentou. Ha 1 clientes esperando.  
Cliente11 acordou o barbeiro  
Barbeiro foi acordado  
[BARBEIRO] Cortando cabelo... ha 0 clientes esperando.  
Sala vazia: barbeiro vai dormir  
Cliente 12 chegou e sentou. Ha 1 clientes esperando.  
Cliente12 acordou o barbeiroBarbeiro foi acordado  
[BARBEIRO] Cortando cabelo... ha 0 clientes esperando.  
  
Sala vazia: barbeiro vai dormir  
Cliente 13 chegou e sentou. Ha 1 clientes esperando.  
ClienteBarbeiro foi acordado  
[BARBEIRO] Cortando cabelo... ha 0 clientes esperando.  
13 acordou o barbeiro  
Sala vazia: barbeiro vai dormir  
Cliente 14 chegou e sentou. Ha 1 clientes esperando.  
Cliente14 acordou o barbeiro  
Barbeiro foi acordado  
[BARBEIRO] Cortando cabelo... ha 0 clientes esperando.  
Sala vazia: barbeiro vai dormir  
Cliente 15 chegou e sentou. Ha 1 clientes esperando.  
ClienteBarbeiro foi acordado  
[BARBEIRO] Cortando cabelo... ha 0 clientes esperando.  
15 acordou o barbeiro  
Sala vazia: barbeiro vai dormir
```

Teste com vários clientes, e um só barbeiro (versão errada)

Código funcional:

Foram utilizados os conceitos de mutex para exclusão mútua e impedir condição de corrida, bem como variáveis de condição para permitir a sincronicidade entre os eventos e o recebimento dos sinais que antes eram perdidos.

```
[BARBEIRO] Corte finalizado para o cliente atual!  
Cliente 4 agradece pelo cabelo cortado!  
[BARBEIRO] Sala vazia. Dormindo.  
Cliente 5 entrou e sentou. (Cadeiras: 1)  
[BARBEIRO] Iniciando corte. Clientes na sala: 0  
[BARBEIRO] Corte finalizado para o cliente atual!  
Cliente 5 agradece pelo cabelo cortado!  
[BARBEIRO] Sala vazia. Dormindo.  
Cliente 6 entrou e sentou. (Cadeiras: 1)  
[BARBEIRO] Iniciando corte. Clientes na sala: 0  
[BARBEIRO] Corte finalizado para o cliente atual!  
Cliente 6 agradece pelo cabelo cortado!  
[BARBEIRO] Sala vazia. Dormindo.  
Cliente 7 entrou e sentou. (Cadeiras: 1)  
[BARBEIRO] Iniciando corte. Clientes na sala: 0  
[BARBEIRO] Corte finalizado para o cliente atual!  
Cliente 7 agradece pelo cabelo cortado!  
[BARBEIRO] Sala vazia. Dormindo.  
Cliente 8 entrou e sentou. (Cadeiras: 1)  
[BARBEIRO] Iniciando corte. Clientes na sala: 0  
[BARBEIRO] Corte finalizado para o cliente atual!  
Cliente 8 agradece pelo cabelo cortado!  
[BARBEIRO] Sala vazia. Dormindo.  
Cliente 9 entrou e sentou. (Cadeiras: 1)  
[BARBEIRO] Iniciando corte. Clientes na sala: 0  
[BARBEIRO] Corte finalizado para o cliente atual!  
Cliente 9 agradece pelo cabelo cortado!  
[BARBEIRO] Sala vazia. Dormindo.
```

Teste com vários clientes, e um só barbeiro (versão correta)

6.6 - Ponte de mão única

Código disfuncional:

Na versão inicial, veículos de sentidos opostos acedem à ponte ao mesmo tempo, causando colisões e mensagens embaralhadas no terminal.

Código funcional:

Para corrigir, utilizou-se `std::mutex` para garantir a exclusão mútua (apenas um carro altera o estado da ponte por vez) e `std::condition_variable` para bloquear o fluxo oposto. Quando a ponte esvazia, os carros do outro lado são notificados para avançar. Criou-se também um limite de passagens consecutivas

para evitar a inanição (starvation), impedindo que um único sentido monopolize a ponte

```
● bash-5.1$ g++ SOa6.cpp -o SOa6_errado -pthread
● bash-5.1$ ./SOa6_errado
[ERRO] Carro [ERRO] Carro 1 (ESQ -> DIR) ENTRANDO na ponte.
7 (DIR -> ESQ) ENTRANDO na ponte.
[ERRO] Carro 8 (DIR -> ESQ) ENTRANDO na ponte.
[ERRO] Carro 3 (ESQ -> DIR) ENTRANDO na ponte.
[ERRO] Carro 9 (DIR -> ESQ) ENTRANDO na ponte.
[ERRO] Carro 4 (ESQ -> DIR) ENTRANDO na ponte.
[ERRO] Carro 2 (ESQ -> DIR) ENTRANDO na ponte.
[ERRO] Carro 10 (DIR -> ESQ) ENTRANDO na ponte.
[ERRO] Carro 5 (ESQ -> DIR) ENTRANDO na ponte.
[ERRO] Carro 11 (DIR -> ESQ) ENTRANDO na ponte.
[ERRO] Carro 6 (ESQ -> DIR) ENTRANDO na ponte.
[ERRO] Carro 12 (DIR -> ESQ) ENTRANDO na ponte.
[ERRO] Carro [ERRO] Carro 31 (ESQ -> DIR) SAINDO da ponte.
(ESQ -> DIR) SAINDO da ponte.
[ERRO] Carro 9 (DIR -> ESQ) SAINDO da ponte.
[ERRO] Carro 4 (ESQ -> DIR) SAINDO da ponte.
[ERRO] Carro 2 (ESQ -> DIR) SAINDO da ponte.
[ERRO] Carro 5 (ESQ -> DIR) SAINDO da ponte.
[ERRO] Carro 6 (ESQ -> DIR) SAINDO da ponte.
[ERRO] Carro 8 (DIR -> ESQ) SAINDO da ponte.
[ERRO] Carro 11 (DIR -> ESQ) SAINDO da ponte.
[ERRO] Carro 10 (DIR -> ESQ) SAINDO da ponte.
[ERRO] Carro 12 (DIR -> ESQ) SAINDO da ponte.
[ERRO] Carro 7 (DIR -> ESQ) SAINDO da ponte.
Simulacao finalizada.
```

6.7 - Estacionamento com vagas limitadas

Código disfuncional:

Na versão inicial, ocorre uma condição de corrida: vários carros verificam que há vagas e entram quase ao mesmo tempo. Como não há proteção, o limite de capacidade do estacionamento é facilmente ultrapassado

.Código funcional:

Para corrigir o problema, usou-se `std::mutex` para garantir a exclusão mútua (apenas um carro altera o contador de vagas por vez) e `std::condition_variable` para colocar os carros em espera quando o

estacionamento lota. Quando um veículo sai, a variável de condição avisa apenas um carro da fila para avançar, garantindo fluxo contínuo e respeitando o limite máximo de vagas.

```
bash-5.1$ g++ S0a7.cpp -o S0a7_errado -pthread
bash-5.1$ ./S0a7_errado
Carro Carro Carro 2 chegou na cancela.
4 chegou na cancela.
Carro 5 chegou na cancela.
Carro 6 chegou na cancela.
Carro 17 chegou na cancela.
   chegou na cancela.
Carro 3 chegou na cancela.
Carro 8 chegou na cancela.
Carro 9 chegou na cancela.
Carro 10 chegou na cancela.
[ERRO] Carro 2 ENTROU. Ocupacao: 1/3
[ERRO] Carro [ERRO] Carro 4 ENTROU. Ocupacao: 3/3
[ERRO] Carro 7 ENTROU. Ocupacao: [ERRO] Carro 59[ERRO] Carro /6[ERRO] Carro 3 ENTROU. Ocupacao: 3
1[ERRO] Carro 8 ENTROU. Ocupacao: 9/ ENTROU. Ocupacao: 39/3
   ENTROU. Ocupacao: 10/3
[ERRO] Carro 10 ENTROU. Ocupacao: 10/3
7/3
[ERRO] Carro 5 ENTROU. Ocupacao: 10/3

   ENTROU. Ocupacao: 10/3
Carro Carro 2 SAIU. Ocupacao: 7/31
   SAIU. Ocupacao: Carro 6 SAIU. Ocupacao: 0/3
Carro 9 SAIU. Ocupacao: 0/3
Carro 3 SAIU. Ocupacao: 0/3
Carro 7 SAIU. Ocupacao: 0/3
Carro Carro 5 SAIU. Ocupacao: 0/3
0/3
8Carro 4 SAIU. Ocupacao: 0/3
   SAIU. Ocupacao: 0/3
Carro 10 SAIU. Ocupacao: 0/3
```

6.8 - Impressora compartilhada

Código disfuncional:

Na versão inicial, uma condição de corrida faz com que os processos tentem acessar a fila ao mesmo tempo e sobrescrevam os trabalhos uns dos outros, perdendo dados.

Código funcional:

Para corrigir, aplicou-se o padrão Produtor-Consumidor. Um `std::mutex` foi usado para proteger a fila, garantindo a exclusão mútua na hora de adicionar trabalhos. Além disso, uma `std::condition_variable` é usada para acordar a impressora apenas quando há novos documentos, garantindo que tudo seja impresso na ordem certa e sem perdas.

```
bash-5.1$ ./50a8_certo
  Pagina 1/2...
[SPOOLER] Processo 9 enfileirou 2 paginas.
[SPOOLER] Processo 4 enfileirou 2 paginas.
  Pagina 2/2...
>> [IMPRESSORA] Processo 2 concluido!

>> [IMPRESSORA] Imprimindo Processo 5...
  Pagina 1/2...
  Pagina 2/2...
>> [IMPRESSORA] Processo 5 concluido!

>> [IMPRESSORA] Imprimindo Processo 6...
  Pagina 1/2...
  Pagina 2/2...
>> [IMPRESSORA] Processo 6 concluido!

>> [IMPRESSORA] Imprimindo Processo 1...
  Pagina 1/2...
  Pagina 2/2...
>> [IMPRESSORA] Processo 1 concluido!

>> [IMPRESSORA] Imprimindo Processo 8...
  Pagina 1/2...
  Pagina 2/2...
>> [IMPRESSORA] Processo 8 concluido!

>> [IMPRESSORA] Imprimindo Processo 7...
  Pagina 1/2...
  Pagina 2/2...
>> [IMPRESSORA] Processo 7 concluido!

>> [IMPRESSORA] Imprimindo Processo 10...
  Pagina 1/2...
  Pagina 2/2...
>> [IMPRESSORA] Processo 10 concluido!

>> [IMPRESSORA] Imprimindo Processo 3...
  Pagina 1/2...
  Pagina 2/2...
>> [IMPRESSORA] Processo 3 concluido!

>> [IMPRESSORA] Imprimindo Processo 9...
  Pagina 1/2...
  Pagina 2/2...
>> [IMPRESSORA] Processo 9 concluido!

>> [IMPRESSORA] Imprimindo Processo 4...
  Pagina 1/2...
  Pagina 2/2...
>> [IMPRESSORA] Processo 4 concluido!

Impressora desligada com sucesso. Fila atendida na ordem correta!
```

